```
"""
SOL/USDT signal bot (Gate.io public API) ->
Telegram alert.
Single-run script: ek baar chalta hai,
signal check karta hai, band ho jata hai.
Scheduler (GitHub Actions cron) isko baar-
baar chalayega.

ENV vars (optional): TG_TOKEN, TG_CHAT_ID
Sirf alerts bhejta hai, trade nahi karta.
Financial advice nahi hai.
"""
import os
import requests

BASE = "https://api.gateio.ws/api/v4"
PAIR = "SOL_USDT"
ATR_SL_MULT = 1.5   # SL = 1.5 x ATR
T1_R, T2_R = 1.5, 3.0
MIN_SCORE = 3       # itna score ho tabhi
LONG/SHORT


def get(path, params=None):
    r = requests.get(BASE + path,
params=params, timeout=15)
    r.raise_for_status()
    return r.json()
```

```
def candles(interval, limit=200):
    raw = get("/spot/candlesticks",
              {"currency_pair": PAIR,
"interval": interval, "limit": limit})
    raw = raw[:-1]  # last candle abhi band
nahi hui, hata do
    # [t, quote_vol, close, high, low,
open, base_vol, closed]
    c = [float(x[2]) for x in raw]
    h = [float(x[3]) for x in raw]
    l = [float(x[4]) for x in raw]
    return h, l, c


def ema(vals, n):
    k = 2 / (n + 1)
    out = [vals[0]]
    for v in vals[1:]:
        out.append(v * k + out[-1] * (1 -
k))
    return out


def atr(h, l, c, n=14):
    tr = [h[0] - l[0]]
    for i in range(1, len(c)):
        tr.append(max(h[i] - l[i], abs(h[i]
- c[i - 1]), abs(l[i] - c[i - 1])))
    out = [tr[0]]
    for t in tr[1:]:
        out.append((out[-1] * (n - 1) + t)
/ n)
    return out
```

```
def rsi(c, n=14):
    gains = losses = 0.0
    for i in range(1, n + 1):
        d = c[i] - c[i - 1]
        gains += max(d, 0)
        losses += max(-d, 0)
    ag, al = gains / n, losses / n
    for i in range(n + 1, len(c)):
        d = c[i] - c[i - 1]
        ag = (ag * (n - 1) + max(d, 0)) / n
        al = (al * (n - 1) + max(-d, 0)) /
n
    return 100.0 if al == 0 else 100 - 100
/ (1 + ag / al)


def supertrend(h, l, c, period=10,
mult=3.0):
    a = atr(h, l, c, period)
    n = len(c)
    fub, flb, st, trend = [0.0] * n, [0.0]
* n, [0.0] * n, [1] * n
    for i in range(n):
        hl2 = (h[i] + l[i]) / 2
        ub, lb = hl2 + mult * a[i], hl2 -
mult * a[i]
        if i == 0:
            fub[i], flb[i], st[i] = ub, lb,
lb
            continue
        fub[i] = ub if (ub < fub[i - 1] or
c[i - 1] > fub[i - 1]) else fub[i - 1]
        flb[i] = lb if (lb > flb[i - 1] or
```

```
c[i - 1] < flb[i - 1]) else flb[i - 1]
        if trend[i - 1] == 1:
            trend[i] = -1 if c[i] < flb[i]
else 1
        else:
            trend[i] = 1 if c[i] > fub[i]
else -1
        st[i] = flb[i] if trend[i] == 1
else fub[i]
    return st[-1], trend[-1]


def funding_rate():
    try:
        d =
get(f"/futures/usdt/contracts/{PAIR}")
        return float(d.get("funding_rate",
0))
    except Exception:
        return None


def long_short_ratio():
    try:
        d =
get("/futures/usdt/contract_stats",
                {"contract": PAIR,
"interval": "1h", "limit": 1})
        v = d[-1].get("lsr_account")
        return float(v) if v is not None
else None
    except Exception:
        return None
```

```
def analyse():
    h, l, c = candles("1h")
    h4, l4, c4 = candles("4h")
    price = c[-1]
    notes, score = [], 0

    e5, e10, e30 = ema(c, 5)[-1], ema(c,
10)[-1], ema(c, 30)[-1]
    if price > e30 and e5 > e10:
        score += 1; notes.append("1h EMA
bullish (+1)")
    elif price < e30 and e5 < e10:
        score -= 1; notes.append("1h EMA
bearish (-1)")
    else:
        notes.append("1h EMA mixed (0)")

    st, tr = supertrend(h, l, c)
    score += 2 * tr
    notes.append(f"SuperTrend {'up' if tr
== 1 else 'down'} @ {st:.2f} ({'+2' if tr
== 1 else '-2'})")

    e30_4h = ema(c4, 30)[-1]
    if c4[-1] > e30_4h:
        score += 1; notes.append("4h trend
up (+1)")
    else:
        score -= 1; notes.append("4h trend
down (-1)")

    r = rsi(c)
    if r > 55:
```

```
        score += 1; notes.append(f"RSI
{r:.0f} strong (+1)")
    elif r < 45:
        score -= 1; notes.append(f"RSI
{r:.0f} weak (-1)")
    else:
        notes.append(f"RSI {r:.0f} neutral
(0)")

    fr = funding_rate()
    if fr is not None:
        if fr > 0.0005:
            score -= 1;
notes.append(f"Funding {fr*100:.3f}%
crowded longs (-1)")
        elif fr < -0.0005:
            score += 1;
notes.append(f"Funding {fr*100:.3f}%
crowded shorts (+1)")
        else:
            notes.append(f"Funding
{fr*100:.3f}% normal (0)")

    lsr = long_short_ratio()
    if lsr is not None:
        if lsr > 1.5:
            score -= 1; notes.append(f"L/S
ratio {lsr:.2f} longs heavy (-1)")
        elif lsr < 0.8:
            score += 1; notes.append(f"L/S
ratio {lsr:.2f} shorts heavy (+1)")
        else:
            notes.append(f"L/S ratio
{lsr:.2f} balanced (0)")
```

```
    a = atr(h, l, c)[-1]
    risk = ATR_SL_MULT * a
    if score >= MIN_SCORE:
        bias = "LONG"
        sl, t1, t2 = price - risk, price +
T1_R * risk, price + T2_R * risk
    elif score <= -MIN_SCORE:
        bias = "SHORT"
        sl, t1, t2 = price + risk, price -
T1_R * risk, price - T2_R * risk
    else:
        bias, sl, t1, t2 = "WAIT", None,
None, None

    return dict(price=price, score=score,
bias=bias, sl=sl, t1=t1, t2=t2,
                atr=a, notes=notes)


def format_msg(x):
    lines = [f"SOL/USDT | Price
{x['price']:.2f} | Score {x['score']}",
             f"Bias: {x['bias']}"]
    if x["sl"]:
        lines += [f"Entry ~
{x['price']:.2f}",
                  f"SL: {x['sl']:.2f}",
                  f"T1: {x['t1']:.2f}
({T1_R}R, 50% book + SL entry par)",
                  f"T2: {x['t2']:.2f}
({T2_R}R)",
                  f"ATR(14):
{x['atr']:.2f}"]
```

```
    lines.append("")
    lines += x["notes"]
    lines.append("\nSirf alert hai, advice
nahi. SL ke bina trade mat lo.")
    return "\n".join(lines)


def send_telegram(text):
    token, chat = os.getenv("TG_TOKEN"),
os.getenv("TG_CHAT_ID")
    if not token or not chat:
        print("Telegram keys nahi mili,
sirf print kar raha hoon.")
        return

requests.post(f"https://api.telegram.org/bo
t{token}/sendMessage",
                  data={"chat_id": chat,
"text": text}, timeout=15)


if __name__ == "__main__":
    result = analyse()
    msg = format_msg(result)
    print(msg)
    # Sirf tab alert jab setup bane (WAIT
par spam nahi)
    if result["bias"] != "WAIT":
        send_telegram(msg)
```